

Cómo se subió Taulamic al servidor

La idea ha sido descargar el proyecto desde GitHub, copiarlo al servidor, preparar dos contenedores uno para la web y otro para la API y conectar el dominio con la aplicación mediante Nginx.

1. Descargar el proyecto

Primero cloné la versión actual del repositorio:

```
git clone https://github.com/quintasc/taulamic.git
```

El proyecto tiene dos aplicaciones:

- apps/web: la parte visual, desarrollada con Next.js.
- apps/api: la API, desarrollada con NestJS.

2. Revisar el servidor

Después entré al servidor por SSH como administrador y comprobé qué había instalado.

El servidor ya tenía:

- Ubuntu
- Docker
- Docker Compose
- Nginx
- Apache
- Plesk

Como Docker ya estaba disponible, lo utilicé para ejecutar el proyecto de forma aislada y mantenerlo separado del resto de aplicaciones alojadas en el servidor.

3. Corregir la carpeta del dominio

Plesk había creado accidentalmente esta carpeta:

```
/var/www/vhosts/alumnesmonlau.com/taulamiq.alumnesmonlau.com
```

Tenía una q al final de taulamiq, pero el nombre correcto es taulamic.

Actualicé la configuración de Plesk y dejé como ruta oficial:

```
/var/www/vhosts/alumnesmonlau.com/taulamic.alumnesmonlau.com
```

Así, aunque Plesk vuelva a regenerar la configuración del dominio, seguirá utilizando la carpeta correcta.

4. Preparar los contenedores

Preparé un contenedor para cada parte del proyecto.

El contenedor de la API:

1. Instala las dependencias de NestJS.
2. Compila el código TypeScript.
3. Arranca la API en el puerto interno 3000.

El contenedor de la web:

1. Instala las dependencias de Next.js.
2. Genera la versión optimizada para producción.
3. Arranca la web en el puerto interno 3001.

Los contenedores solamente publican sus puertos dentro del propio servidor:

127.0.0.1:3100 → API
127.0.0.1:3101 → Web

Esto significa que no están expuestos directamente a Internet. Las peticiones siempre pasan primero por Nginx.

También se configuró Docker para reiniciar automáticamente los contenedores si se detienen o si el servidor se reinicia.

5. Conservar los datos

La API guarda información y archivos en varias carpetas, por ejemplo:

- Eventos
- Invitados
- Planos de salones

Para que estos datos no desaparezcan al actualizar o reconstruir los contenedores, creé un volumen persistente de Docker.

En otras palabras: el programa se puede sustituir por una versión nueva sin borrar los datos creados por los usuarios.

6. Copiar y arrancar el proyecto

Comprimí el proyecto, excluyendo archivos que no eran necesarios, como:

- El historial local de Git
- Dependencias instaladas
- Compilaciones antiguas
- Archivos temporales

Después copié el paquete al servidor mediante SSH, lo descomprimí dentro de la carpeta del dominio y ejecuté:

```
docker compose -f compose.production.yml up -d --build
```

Este comando:

- Construye la web.
- Construye la API.
- Crea la red interna.
- Crea el almacenamiento persistente.
- Arranca ambos contenedores en segundo plano.

La versión que quedó desplegada corresponde al commit:

d66a1f8

7. Conectar el dominio con la aplicación

El dominio lo gestiona Plesk y las peticiones HTTPS llegan primero a Nginx.

Configuré Nginx para que las peticiones normales se envíen a la web:

<https://taulamic.alumnesmonlau.com>

↓
Web Next.js, puerto 3101

Las llamadas realizadas desde la web a `/api/v1` pasan internamente desde Next.js hasta el contenedor de la API.

También dejé una regla directa para la documentación OpenAPI:

<https://taulamic.alumnesmonlau.com/api/docs>

↓
API NestJS, puerto 3100

Se aumentó el tiempo máximo de espera del proxy porque el motor que calcula la distribución de invitados puede tardar varios minutos.

8. Comprobaciones finales

Finalmente comprobé que todo respondiera desde fuera del servidor:

- [Página principal](#) → 200 OK
- [API](#) → 200 OK
- [Documentación OpenAPI](#) → 200 OK
- Certificado HTTPS → válido
- DNS → apunta a 212.227.227.190
- Contenedor web → activo
- Contenedor API → activo
- Configuración de Nginx → válida